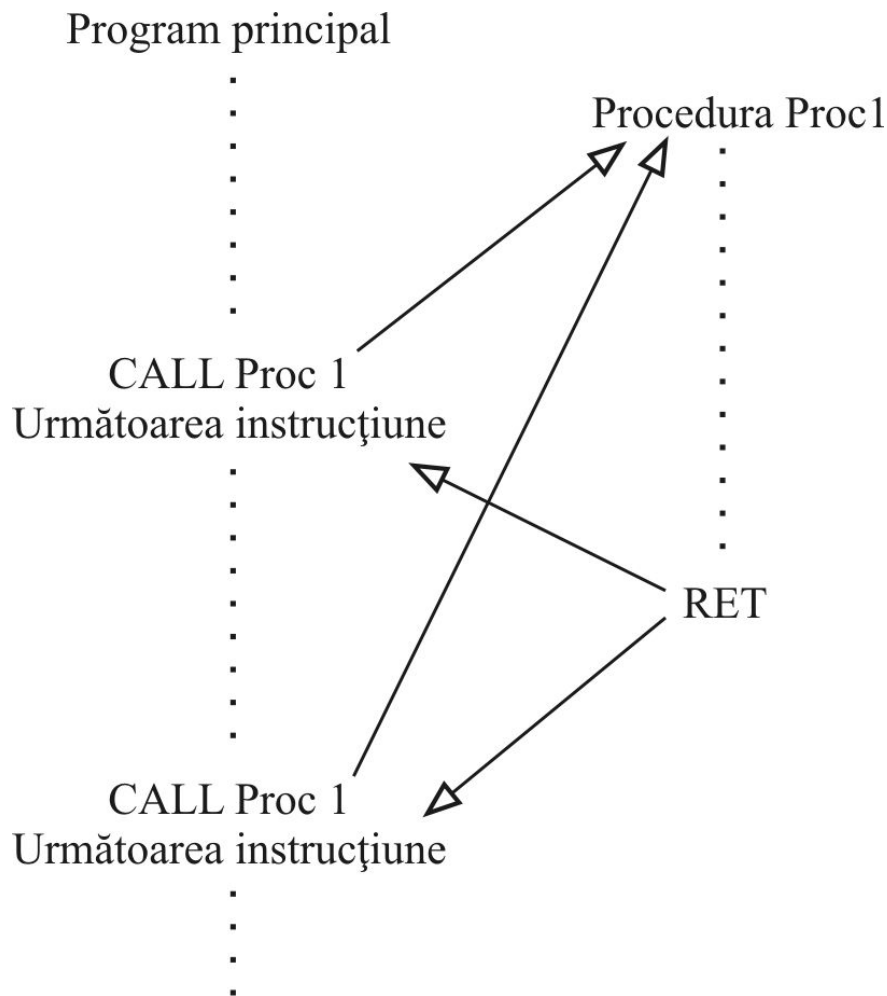


Proiectarea setului de instrucțiuni

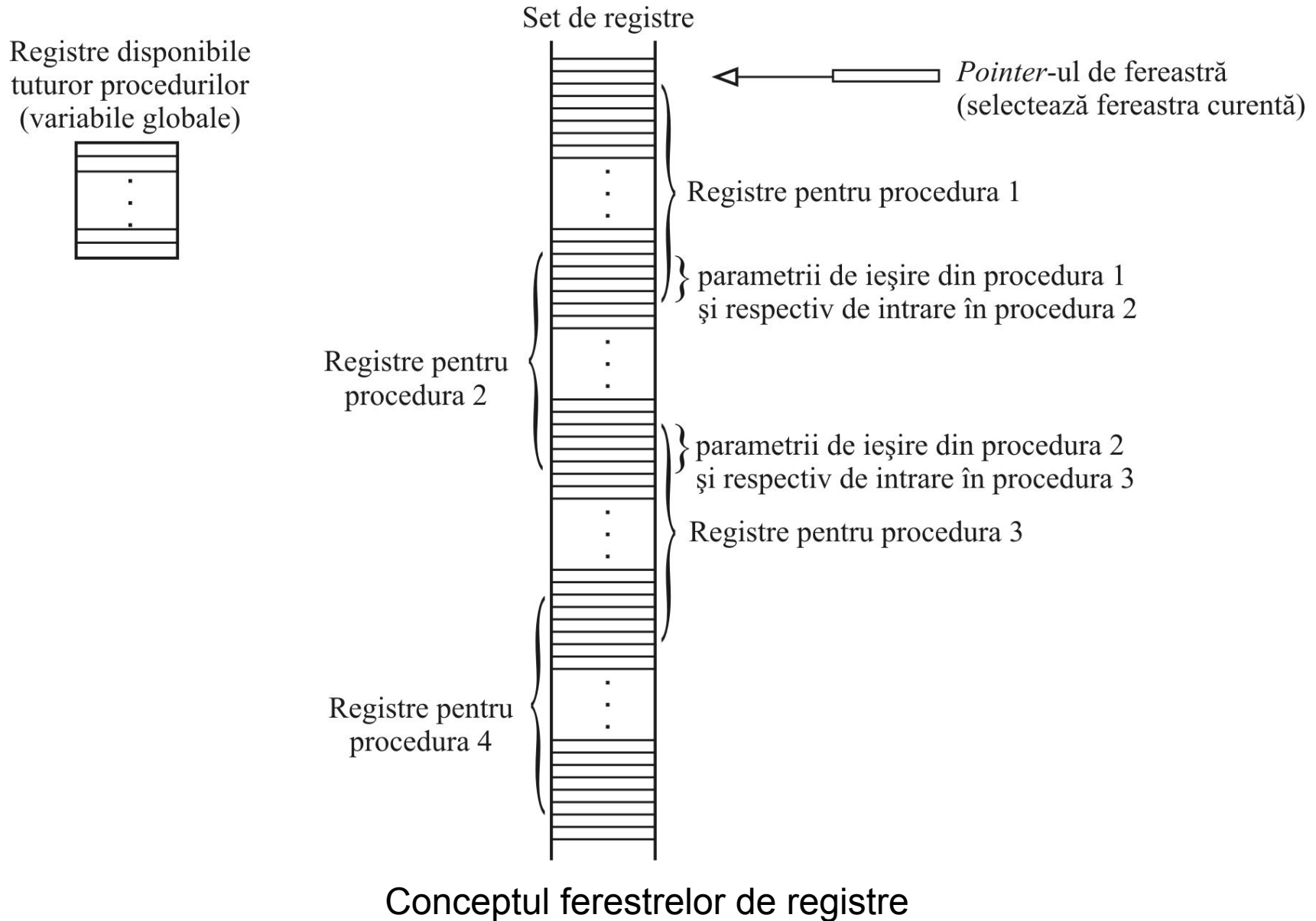
5. Apelul procedurilor (soluția CISC)



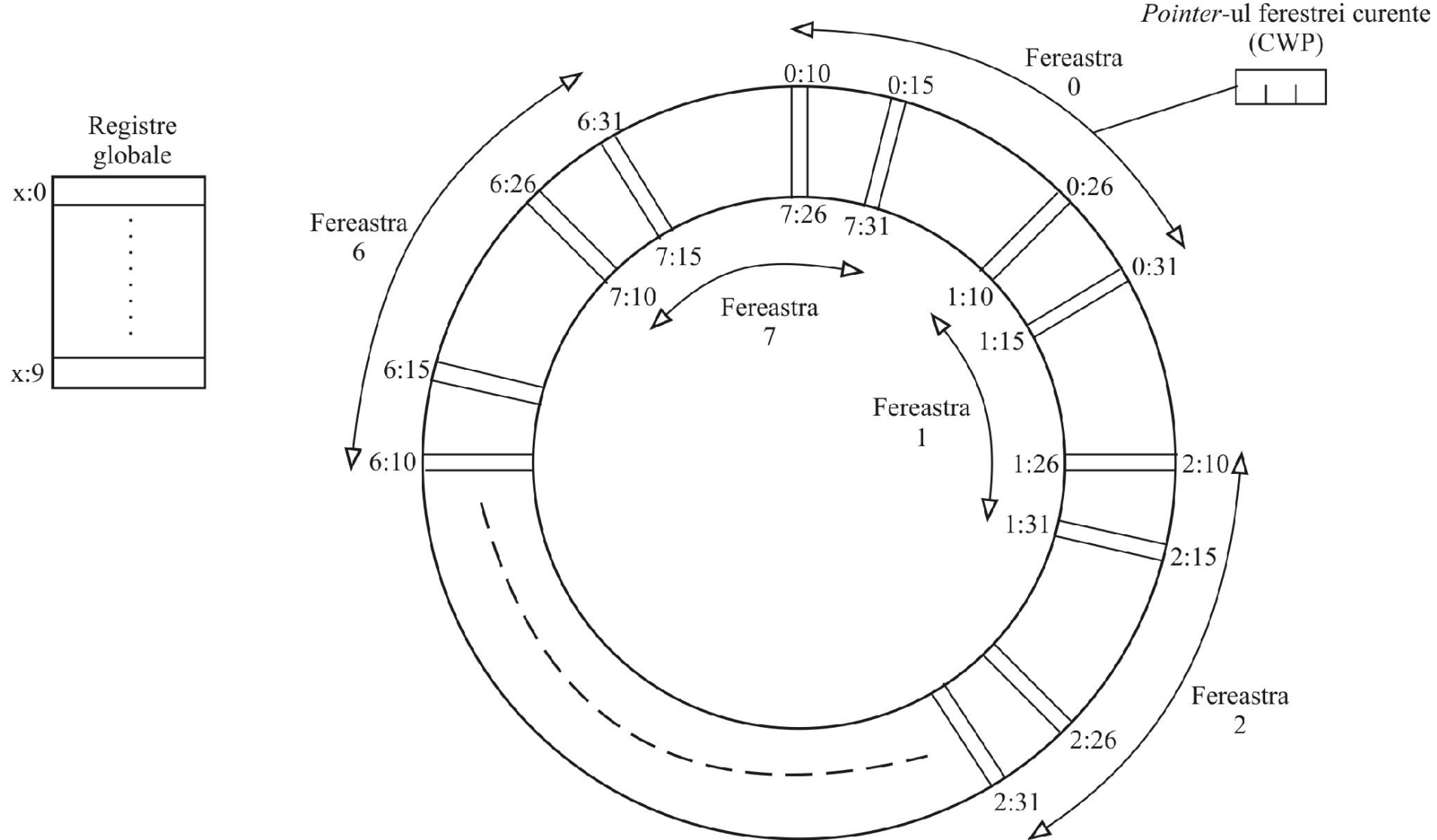
Apel de procedură (CALL) și revenire în apelant (RET)

Proiectarea setului de instrucțiuni

5. Apelul procedurilor (soluție RISC: conceptul *register window*)



5. Apelul procedurilor (soluția RISC II – Univ. Berkley)



Configurația circulară a ferestrelor de registre la procesorul RISC II

5. Apelul procedurilor (soluția clasică RISC: JAL – *jump and link*)

Registre specializate (dedicate mecanismului de apel/revenire):

R31 – *link register*

R29 – *stack pointer*

Secvența de cod necesară pentru apel și revenire:

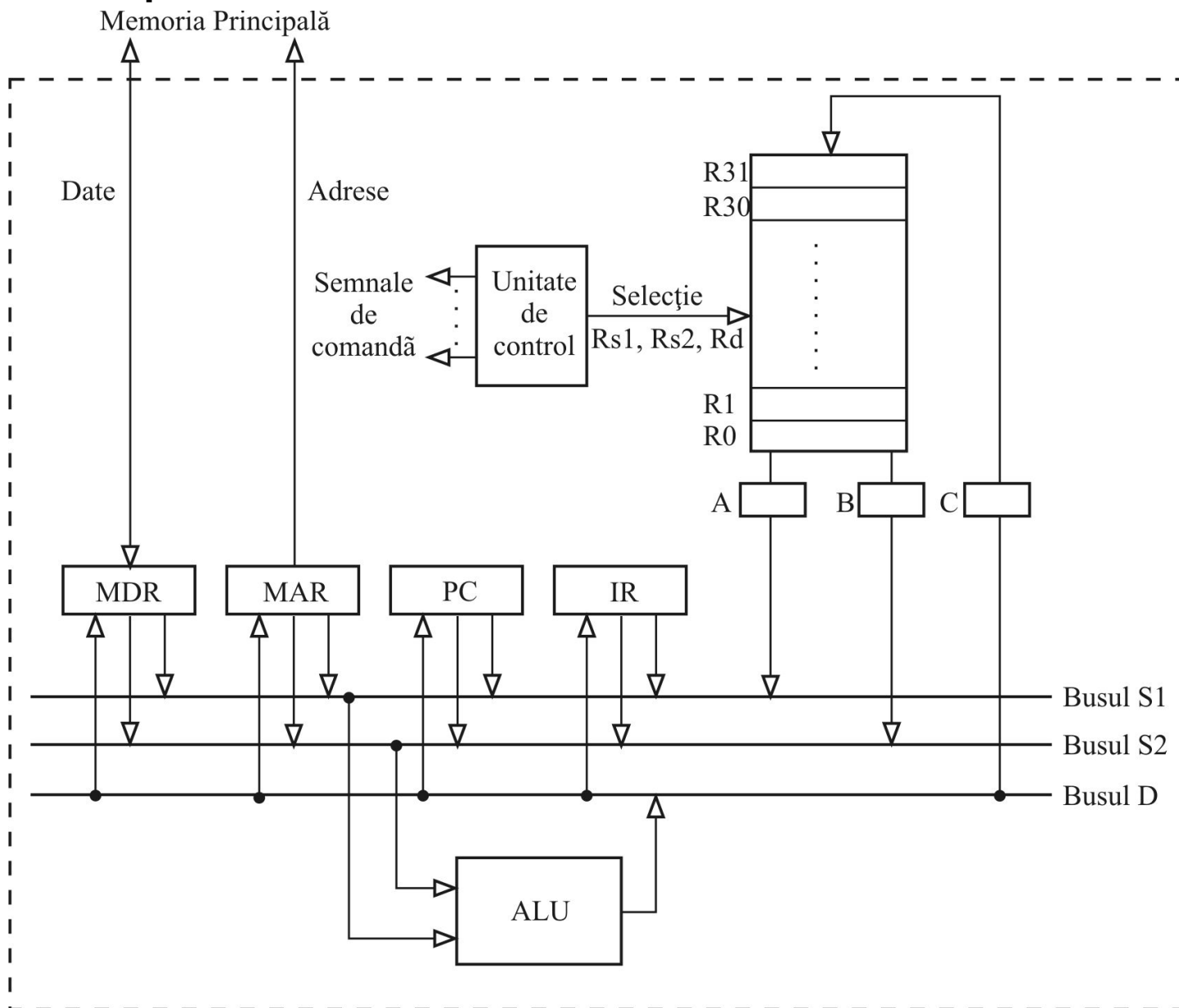
```
SUB R29,R29,4 ; decrementează pointerul de stivă (4 octeți)
ST [R29],R31 ; memorează ultima adresă de revenire în stivă
JAL Proc_Label ; salt la Proc_Label și memorează adresa de
; revenire în R31.
LD R31,0[R29] ; după revenirea din procedură, se restaurează
; adresa de revenire anterioară în R31.
ADD R29,R29,4 ; incrementează pointerul de stivă (4 octeți)
```

Instrucțiunea de revenire (în apelant) de la sfârșitul procedurii:

```
JMP [R31] ; salt la locația a cărei adresă se află în R31
```

Execuția instrucțiunilor

1. Structura procesorului



2. Ciclul *fetch* instrucțiune:

$MAR \leftarrow PC$; se încarcă în MAR adresa instrucțiunii

Memory Read ; citire din memorie

$IR \leftarrow MDR$; se transferă în IR codul instrucțiunii citit din memorie

$PC \leftarrow PC + 4$; incrementare PC (codul instrucțiunii = 4 octeți)

Evidențiind operațiile simultane, ciclul *fetch* instrucțiune devine:

$MAR \leftarrow PC$

Memory Read

$IR \leftarrow MDR$, $PC \leftarrow PC + 4$

3. Ciclul de execuție (partea 1 – independentă de tipul instrucțiunii)

În paralel cu decodificarea instrucțiunii, operanzii preluați din registrele sursă specificate în instrucțiune sunt transferați în registrele tampon A și respectiv B:

$A \leftarrow Rs1$

$B \leftarrow Rs2$

Dacă setul RG este dotat cu doua porturi de citire, atunci cele 2 operații pot fi executate simultan (pas anticipativ, independent de codul instrucțiunii):

$A \leftarrow Rs1$, $B \leftarrow Rs2$

3. Ciclul de execuție (partea a 2-a – dependentă de tipul instrucțiunii)

A. Instrucțiunile aritmetico-logice R-R-R

Execuția instrucțiunilor aritmetico-logice, cum ar fi `ADD R1,R2,R3`, presupune în continuare (după pasul anticipativ) următoarele operații:

$$C \leftarrow A <\text{operație}> B$$
$$Rd \leftarrow C$$

B. Instrucțiunile aritmetico-logice R-R-I

Utilizând notația IR_{15-0} pentru a specifica constanta, după pasul anticipativ, instrucțiunile R-R-I revendică:

$$C \leftarrow A <\text{operație}> IR_{15-0}$$
$$Rd \leftarrow C$$

3. Ciclul de execuție (partea a 2-a – dependentă de tipul instrucțiunii)

C. Instrucțiunile cu referire la memorie (*load/store*) (versiunea 1 de format)

Pentru *load*:

$MAR \leftarrow A + IR_{15-0}$; pregătirea adresei de acces la memorie
Memory Read ; citire din memorie
 $C \leftarrow MDR$; încărcarea datelor citite din memorie
 $Rd \leftarrow C$; transferul datelor citite în Rd

Pentru *store* (prima soluție) :

$MAR \leftarrow A + IR_{15-0}$; pregătirea adresei de acces la memorie
 $B \leftarrow Rd$; acționează *hardware* -ul de redirectare
 $MDR \leftarrow B$; pregătirea datelor de scris în memorie
Memory Write ; scriere în memorie

Pentru *store* (a doua soluție) :

$A \leftarrow Rs1$, $B \leftarrow Rs2$, $C \leftarrow Rd$; pas anticipativ modificat pentru a elimina
; operația $B \leftarrow Rd$ de la prima soluție (crește ;
performanța, dar RG cu 3 porturi de citire!)

$MAR \leftarrow A + IR_{15-0}$; pregătirea adresei de acces la memorie
 $MDR \leftarrow C$; pregătirea datelor de scris în memorie
Memory Write ; scriere în memorie

3. Ciclul de execuție (partea a 2-a – dependentă de tipul instrucțiunii)

C. Instrucțiunile cu referire la memorie (*load/store*) (versiunea 2 de format)

Pentru *load*:

$MAR \leftarrow A + IR_{10-0}$; pregătirea adresei de acces la memorie
Memory Read ; citire din memorie
 $C \leftarrow MDR$; încărcarea datelor citite din memorie
 $Rd \leftarrow C$; transferul datelor citite în Rd

Execuție este identică cu cea de la versiunea 1 de format, doar *offset*-ul (indexul) este codificat pe 11 biți (IR_{10-0}) și nu pe 16 biți (IR_{15-0}).

Pentru *store*:

$MAR \leftarrow A + IR_{10-0}$; pregătirea adresei de acces la memorie
 $MDR \leftarrow B$; pregătirea datelor de scris în memorie
Memory Write ; scriere în memorie

Execuție optimizată (performantă), care nu necesită *hardware* de redirectare sau RG cu trei porturi de citire. Dezavantajul este că *offset*-ul (indexul) este codificat doar pe 11 biți (IR_{10-0}) și nu pe 16 biți (IR_{15-0}).

3. Ciclul de execuție (partea a 2-a – dependentă de tipul instrucțiunii)

D. Instrucțiunile de salt condiționat (*branch*)

Condiție $\leftarrow A <\text{operație}> B$; generare condiție de salt
ALU_R $\leftarrow PC + IR_{15-0}$; generare adresă de salt
IF Condiție=TRUE *then* PC $\leftarrow$ ALU_R ; încărcarea adresei de salt în PC
; (dacă condiția testată este îndeplinită)

E. Instrucțiunile de salt necondiționat (*jump*)

Jump direct (exemplu JMP Label1):

PC $\leftarrow PC + IR_{25-0}$; PC $\leftarrow$ adresa de salt

Jump indirect (exemplu JMP123[R4]):

PC $\leftarrow A + IR_{15-0}$; PC $\leftarrow$ adresa de salt

F. Instrucțiunea JAL (*jump and link*) - pentru apelul procedurilor

R31 $\leftarrow PC$; R31 $\leftarrow$ adresa de revenire
PC $\leftarrow PC + IR_{25-0}$; PC $\leftarrow$ adresa de salt